

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 4: **Database Design, Migrations, and Seeding**
Schema, Schema Change History, and Query Speed

What You'll Learn

1. Explain the **schema** concept and table relationships through **foreign keys**
2. Understand **migrations** as a replayable history of schema changes, and **seeders** for filling in large amounts of initial data
3. Explain the **index** concept and how to verify its impact on query speed

This slide deck covers concepts. Designing the schema, writing migrations, and seeders for Simple POS happens in the practicum jobsheet.

Part 1

A Warehouse With a Catalog vs. Without One

A warehouse with a card catalog

- Every item sits on a labeled shelf
- The catalog records which shelf holds what
- Looking for an item: read the catalog, go straight to the right shelf

A warehouse without one

- The item is still there, somewhere
- Searching means walking every shelf one by one
- In a large warehouse, this eats a lot of time

A database table's schema is the shelf layout itself; an index is the card catalog. Without the right index, the database engine can still find the data, just by scanning the entire table one row at a time.

Schema: A Table's Structural Design

Schema: the design of a table's structure: column names, data types, and constraints (nullable, default, unique) that apply to every row in it.

- A schema gets sketched on paper before a single migration file gets written
- Every column has a data type (number, text, date) and constraints that keep the data valid

Foreign Key: Keeping Relationships Consistent

Foreign Key: a column on one table pointing to the `id` of a row on another table, guaranteeing a row can never point at a parent row that doesn't exist.

authors (one)



articles (many)

- One `authors` row has many `articles`; every `articles` row stores an `author_id` pointing back to its author

Example Schema: A One-to-Many Relationship

Table	Main Columns
authors	id , name
articles	id , author_id (fk), title , published_at
comments	id , article_id (fk), body

- One authors row has many articles
- One articles row has many comments
- The relationships are simple enough to sketch on paper before writing any code

Values That Are Stored, Not Recomputed

- Some columns deliberately store a value as of one point in time, instead of recomputing it every time it's read
- Example: a comment count at the moment an article was published, or a price as it stood when a transaction happened
- That value has to stay exactly what it was at that moment, even if the source data changes later

The validation & security meeting later covers why a value like this must still be recalculated on the server when it's saved, not just trusted from input.

Writing a Migration for a Table

```
Schema::create('articles', function (Blueprint $table) {  
    $table->id();  
    $table->foreignId('author_id')->constrained();  
    $table->string('title');  
    $table->timestamps();  
});
```

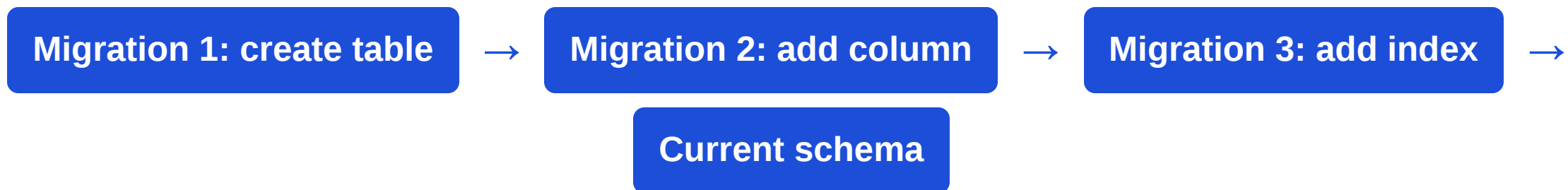
- `foreignId('author_id')` creates an `author_id` column as an unsigned big integer
- `->constrained()` adds a foreign key pointing to `authors.id`, following Laravel's naming convention
- This constraint protects data integrity, but doesn't necessarily mean the column already has an index (see Part 3)

Part 2

Schema change history, and filling in initial data

Migrations: A Replayable Schema History

Migration: a PHP file describing a structural change to a table (creating, altering, or dropping columns) programmatically, so the database schema has a history that can be replayed on any other machine.



Similar to a git commit history for code: a migration is a commit history for the database schema, run in order according to its filename timestamp.

The Golden Rule: Don't Edit, Add a New One

If the schema needs to change, create a new migration, don't edit an old migration that's already run. An old migration edited after it has run leaves the schema history on another machine out of sync with the history on yours.

- Treat migrations like version control for the schema: a new change always means a new migration file

Seeders: Filling in Initial Data

Seeder: a PHP class that fills a table with initial or test data, run through `db:seed` or as part of `migrate:fresh --seed`.

- Useful for sample data during development, and realistic-scale test data for practicing performance work

Seeding at Scale: Row-by-Row vs. Bulk Insert

`Model::create()` in a loop

- One `INSERT` query per row
- Overhead: building an Eloquent object, firing model events
- Slow for hundreds or thousands of rows

`DB::table()->insert()` in batches

- One `INSERT` statement for many rows at once
- Far fewer round-trips to the database
- `array_chunk()` splits the batch because some database engines cap the number of rows per `INSERT`

```
$articles = [];  
foreach ($authorIds as $authorId) {  
    for ($i = 0; $i < 30; $i++) {  
        $articles[] = [  
            'author_id' => $authorId,  
            'title' => fake()->sentence(),  
            'created_at' => now(),  
            'updated_at' => now(),  
        ];  
    }  
}
```

Consistency Across Tables: `DB::transaction()`

When one operation touches more than one table that must stay consistent with each other (e.g. saving an order along with its line items), wrap it in a single `DB::transaction(function () { ... })`. If the process fails partway through, every change inside that block rolls back together, so the data never ends up half-saved.

Part 3

Proving the impact with EXPLAIN QUERY PLAN

Index: A Data Structure for Fast Lookups

Index: an additional data structure the database engine builds over one or more columns, speeding up lookups on that column without scanning the entire table.

Without an index (SCAN)

- The database engine checks every row one by one
- Time grows linearly as the table grows

With an index (SEARCH)

- The database engine jumps straight to the relevant rows
- Barely affected by table size

`constrained()` Doesn't Always Mean There's an Index

- On some database engines (e.g. MySQL), `foreignId()->constrained()` automatically creates an index as a side effect
- On SQLite, `constrained()` only adds a foreign key constraint, not an index
- The constraint protects data integrity (rejecting inserts that point at a nonexistent row), but doesn't speed up lookups

Proving It With `EXPLAIN QUERY PLAN`

- `EXPLAIN QUERY PLAN` : a SQLite command that shows the strategy the database engine will use to run a query, without actually running it

```
sqlite3 database/database.sqlite \  
"EXPLAIN QUERY PLAN SELECT * FROM articles WHERE author_id = 3;"
```

- **Before an index:** the result includes `SCAN articles`
- **After an index is added:** the result changes to `SEARCH articles USING INDEX articles_author_id_index`

If the `sqlite3` command isn't found, the alternative path is `php artisan tinker` with `DB::select("EXPLAIN QUERY PLAN ...")`.

An Index Isn't Premature Optimization

On a table with a few dozen rows, the difference between SCAN and SEARCH is barely noticeable. On a table with thousands of rows, SCAN means every row gets examined on every single query, and that time grows linearly as the table grows. An index isn't premature optimization here; it's a fix for a performance bug that's already real the moment data approaches production scale.

Applying This to Simple POS

- You'll apply the schema, migration, and index concepts directly to the Simple POS case study in the practicum jobsheet
- Designing the category-product-transaction tables, writing a seeder at hundreds-to-thousands-row scale, and proving the index with `EXPLAIN QUERY PLAN`

Full code: github.com/se-polinema/simple-pos , branch `chapter-04`

Summary

- A table's schema is defined through migrations; a migration is a history of changes that can be replayed on another machine, and the golden rule is: add a new migration, never edit an old one
- Foreign keys keep one-to-many relationships between tables consistent; seeders fill in initial data, and large-scale seeding uses `DB::table()->insert()` in batches, far more efficient than `Model::create()` one row at a time
- An index speeds up lookups on frequently filtered columns; a foreign key constraint doesn't automatically mean there's an index, depending on the database engine in use
- `EXPLAIN QUERY PLAN` proves an index's impact directly: from `SCAN` (scanning every row) to `SEARCH` (jumping to the relevant rows)

References & Discussion

Official Laravel documentation (Migrations, Query Builder, Seeding)

Full code: `github.com/se-polinema/simple-pos`

Next meeting: ORM & Data Relations